UAlg
UNIVERSIDADE DO ALGARVE

POO - Programação Orientada por Objetos
OOP - Object-oriented programming

2025/2026

José Valente de Oliveira
jvo@ualg.pt



1

Para quê tanta coisa para escrever uma string na consola?

```
class A {
        public static void main(String [] a) {
            System.out.println("Hello World");
        }
}
```

http://tiny.cc/poo2025-26

2

Problem 1: Distance between 2 points of R^2

Write a program that, given any two points and compute the integer part of the distance between them.

Let the coordenates of points *A*, and *B* be (x_A, y_A) e (x_B, y_B) respectively, the distance *d* between *A* and *B* is given by:

$$d = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Input

Input has two rows. The first has two integers x_A, y_A, representing the coordinates of point *A*. The second line is similar to the first one, representing the coordinates of point *B*. -10000 <= x_A <= 1000, -1000 <= y_A <= 1000

Output

The output is an integer denoting the integer part of the distance between the input points

Sample input	**Sample output**
1 1 4 2	3

http://tiny.cc/poo2025-26

3

A *very bad* solution *a la* C

```
// A very poor solution, useful for motivation only

public class VeryBad {
    public static void main (String [] args) {
        int xA, yA, xB, yB;
        Scanner sc = new Scanner (System.in);
        xA = sc.nextInt();
        yA = sc.nextInt();
        xB = sc.nextInt();
        yB = sc.nextInt();
        System.out.print ( (int) dist(xA, yA, xB, yB) );
        sc.close();
    } }
```


4

A *very bad* solution *a la* C

```
// A very poor solution, useful for motivation only

  public static double dist(int x1, int y1, int x2, int y2)  {
    int dx = x1 – x2;
    int dy = y1 – y2;
    return Math.sqrt(dx*dx + dy*dy);
  }
}  // end of class VeryBad
```


5

Problem 1: Distance between 2 points of R^2

- Uma das soluções mais simples seria:

```
// File: Client.java

Cria a classe Client

    Cria o objeto ponto a
    Cria o objeto ponto b

    Pede a a para calcular a distância
      entre si próprio e b
```

```
// File: Client.java
import java.util.Scanner;
public abstract class Client {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Ponto a = new Ponto();
    Ponto b = new Ponto();
    a.x = sc.nextInt();
    a.y = sc.nextInt();
    b.x = sc.nextInt();
    b.y = sc.nextInt();
    System.out.println((int) a.dist(b));
  } }
```


6

Problem 1: Distance between 2 points of R^2

- Uma das soluções mais simples seria:

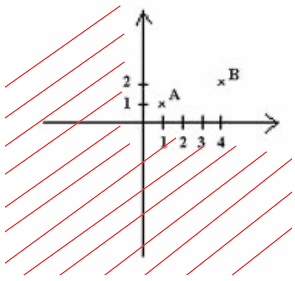
```
// File: Client.java
import java.util.Scanner;
public abstract class Client {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     Ponto a = new Ponto();
     Ponto b = new Ponto();
     a.x = sc.nextInt();
     a.y = sc.nextInt();
     b.x = sc.nextInt();
     b.y = sc.nextInt();
     System.out.println((int) a.dist(b));
} }
```

```
// File:  Ponto.java
class Ponto {
  int x, y;
  double dist (Ponto that) {
       int dx = this.x - that.x;
       int dy = this.y - that.y;
       return Math.sqrt(dx*dx + dy*dy);
   }
}
```

$$d = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

7

Problem B: Distance between points within the first quadrant



Write a program that, given any two points **with non-negative coordinates**, compute the integer part of the distance between them

8

Again: a ***very bad*** solution *a la* C

```
public class VeryBad {
    public static void main (String [] args) {
        int xA, yA, xB, yB;
        Scanner sc = new Scanner (System.in);
        xA = sc.nextInt();
        //... Other inputs omitted for brevity

        if ((xA <0) || (yA<0))
                System.exit(0);

        xA = -5; // Clear violation and undetected by the compiler! ☹
        //... as before....
```


9

The simplest class implementation no longer valid **B**

```
/** @version  0.0 */
// No longer valid for B; useful for motivation only
// Why???
class Point {
    int x, y;
    double dist(Point p) {
        int dx = x - p.x;
        int dy = y - p.y;
        return Math.sqrt(dx*dx + dy*dy);
    }
}
```


10

The simplest class implementation no longer valid **B**

```
// File: Client.java
import java.util.Scanner;
public abstract class Client {
  public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
     Ponto a = new Ponto();
     Ponto b = new Ponto();
     a.x = sc.nextInt();
     //... Other inputs omitted for brevity
      if ((xA <0) || (yA<0))
                  System.exit(0);

    a.x = -5; // Clear violation and undetected by the compiler! ☹
     System.out.println((int) a.dist(b));
  }
```

11

A (much) better version with visibility control

```
@version  1.0
class Point {
   private int x, y;
   public double dist(Point p) {
         int dx = x – p.getX();
         int dy = y – p.getY();
         return  Math.sqrt(dx*dx+dy*dy);
   }
   //… to be completed
}
```


12

A client of Point

```
public class Main {
    public static void main(String[] args) {

        Point a = new Point();

        a.x = -1; // Compile time ERROR: Access violation

    }
}
```



13

What we would like for the client of Point

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);

        Point a = new Point(sc.nextInt(), sc.nextInt());
        Point b = new Point(sc.nextInt(), sc.nextInt());
        System.out.println( (int) a.dist(b) );
        sc.close();
    }
}
```

```
// A very poor solution, usefull for motivation only

public class VeryBad {
    public static void main (String [] args) {
        int xA, yA, xB, yB;
        Scanner sc = new Scanner (System.in);
        xA = sc.nextInt();
        yA = sc.nextInt();
        xB = sc.nextInt();
        yB = sc.nextInt();
        System.out.print ( (int) dist(xA, yA, xB, yB) );
        sc.close();
    }}
```


14

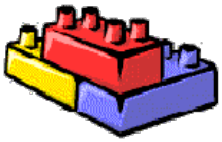
Classes and objects

- A class can be seen as a description of a set of objects sharing the same data structure (not the same data) and the same behaviour (functions)



15

What's an object?



- An object represents a concrete entity of the problem domain, that handles a specific task
- Examples include:
  - Tangible things as cars, bank accounts
  - Roles as employee, boss, ...
  - Interactions as contract, sale, ...
- An object is like a black box: Internal details are hidden.

16

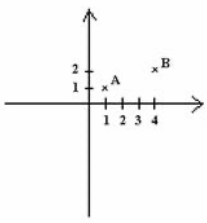
What's an object?

- An object can be seen as an abstraction which represents an entity with interest for the problem to solve.
- An object has:
  - State
  - Behaviour
  - Identity

http://tiny.cc/poo2025-26

17

What's an object?



- An object has:
  - State
  - Behaviour
  - Identity

Point *A* is an *object* with*:*

-State: Coordinates (X_A=1, Y_A=1)

- Behaviour: computes the distance between itself and another point

- Identify: Object point *A* is different from any other object

http://tiny.cc/poo2025-26

18

What's an object?

Objects of the class Rectangle describe rectangular shapes such as those in the figure



© sinankocasian/iStockphoto.

19

What's an object?

A `Rectangle` object is implemented as a block of memory that, in this naïve example, holds four numbers, the *state* of the object

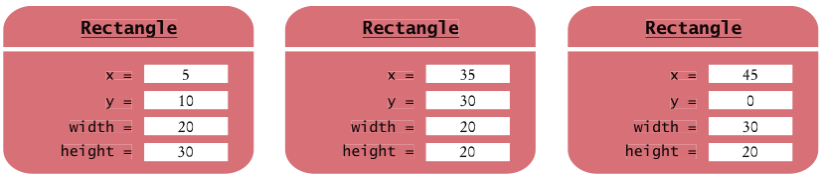


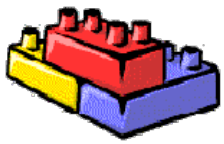
Figure 11 Rectangle Objects

20

Yet some more examples of classes and their objects

Class	Objects	Data	Typical functions
Point	(0, 0) (4.5; 7.2)	x=0; y =0 x=4,5; y=7.2	dist
BankAccount	account1	Balance=0 Number=12	transfer withdraw
Fraction	½	Numerator = 1 Denominator =2	sum

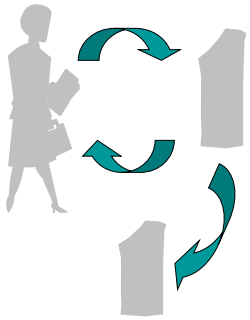
OOP basics



- Three basic ideas:
  - Identifying *objects* and assigning *responsibilities* to these objects.
  - Build complex objects based on simple ones
  - Objects communicate with other objects by sending *messages*.

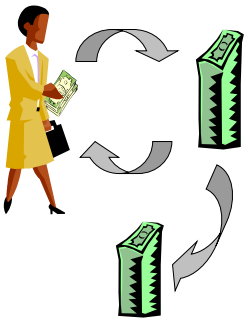
Procedural vs. Object-Oriented

- Procedural



Focus on procedures: credit, debit, compute interest

- Object Oriented

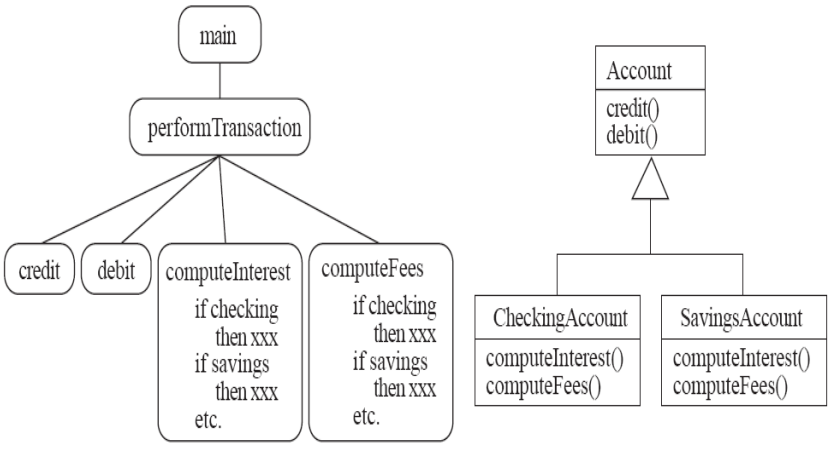


Focus on entities with a well-defined role in the problem domain: Customer, Account

http://tiny.cc/poo2025-26

23

Procedural vs. Object-Oriented



http://tiny.cc/poo2025-26

24

Why should we care about OOP?

- Modularity - large software projects can be splited into independent, self-contained modules
- Reusability - New software components can be written or developed from existing ones
- Extensibility – New software can be developed simply by adding to existing one

http://tiny.cc/poo2025-26

25

Course main topics

- Elementary concepts of OOP
- Introduction to Object Oriented Modelling and UML (Unified Modelling Language)
- Principles, concepts and techniques of OOP
- Object Oriented Programming using Java
- Elementary data structures
- Fundamental Design Patterns

http://tiny.cc/poo2025-26

26

Learning outcomes

1. Enumerate, describe, and justify the object-oriented concepts, principles, and techniques
2. Application modeling using UML, emphasizing architectural aspects (diagrams of classes and objects)
3. Select and employ fundamental design patterns
4. To use JAVA as an object-oriented programming language

http://tiny.cc/poo2025-26

27

Competências prévias recomendadas

- Programação Imperativa
- Laboratório de Programação
- Algoritmos e Estruturas de Dados

http://tiny.cc/poo2025-26

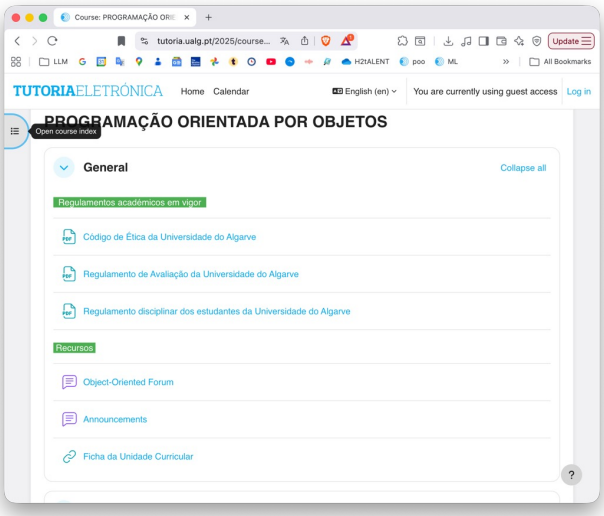
28

Syllabus

1. Object-Oriented concepts and principles;
2. Introduction to Object Oriented Modelling and UML
3. Designing classes
4. Inheritance, polymorphism, and binding
5. Interfaces
6. Exceptions and Error Handling
7. Nested classes
8. Fundamental Design Patterns: Template Method, Strategy, Iterator, Composite, Factory Method
9. Generics
10. Java Collection Framework
11. Stream processing
12. GUI
13. Internet networking (java.net)
14. Reflection

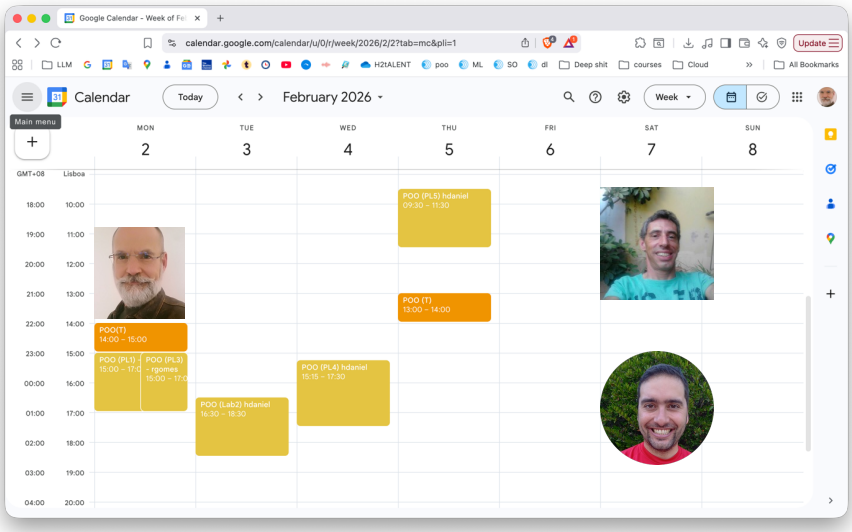
29

tutoria.ualg.pt | http://tiny.cc/poo2025-26



30

Schedule



http://tiny.cc/poo2025-26

31

Schedule: check it weekly !

https://ualgnet.ualg.pt/aplicacoes/horarios

Horário de dúvidas:
imediatamente após as aulas

http://tiny.cc/poo2025-26

32

Textbook

(Horstmann, 2018)

C. Horstmann, Big Java: Early Objects 7th ed, Wiley, December 2018. ISBN: 978-1-119-49909-1

Student companion site

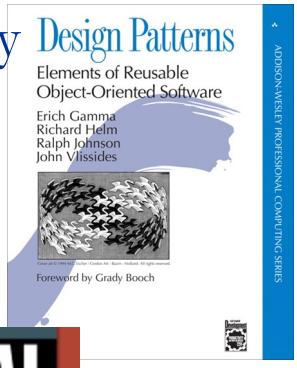


Acknowledgement: Many figures in these slides are from this book

33

Suplementary bibliography

- [Supplementary readings on Design patterns]
  Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Pattern – Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995



- [Supplementary readings on UML]
  UML Quick Ref. www.uml.org



34

Procedimento de avaliação

- Nota final = 0,6 Nota de Exame +
  0,4 Nota Avaliação Prática
- Arredondamentos só na nota final.

35

Admissão a exame

- Todos os alunos regularmente inscritos à unidade curricular serão admitidos a exame (independentemente da época) desde que satisfaçam cumulativamente as seguintes condições:
  - Nota de avaliação prática >= 7,5 valores

36

Avaliação teórica

- Exames (independentemente da época)

  Os exames consistem numa prova escrita sem consulta

37

Avaliação prática

- A nota da componente prática corresponde à média ponderada das notas dos trabalhos práticos.
- Os trabalhos são realizados individualmente ou em grupo.
- A inscrição num grupo de trabalho é obrigatória (só são aceites trabalhos de estudantes inscritos em grupos). Os grupos têm no máximo 3 estudantes.
- Quando um trabalho for realizado em grupo, a nota do trabalho é convertida numa nota individual no momento da validação do trabalho.

38

Assignments' assessment

- Some assignments require only the submission of source code to Mooshak
- Some problems and project require also the submission of a report in pdf format
- Mooshak has submission deadlines
- After a deadline it is not possible to submit assignments
- Excluding Projects, assignments not validated during discussion can be resubmitted to Mooshak and discussed again

if within the submission deadline

http://tiny.cc/poo2025-26

39

Assignments' assessment

- An assignment will be validated if:
  - **fulfil all the requirements and constraints**
  - accepted by Mooshak
  - successfully defended
- **be prepared, especially for Projects:**

  During defence can be asked any questions about the implementation,

  implementation decisions,

  implementation alternatives, etc.

  failing to answer those questions invalidates the assignment

http://tiny.cc/poo2025-26

40

Assignments' assessment criteria

- Criteria for assessment of the assignments:

- Correction	fulfil all requirements features that exceed specifications will not be considered for assessment
- OOP	Implemented according to OOP best practices and design patterns
- Efficiency	use of resources: CPU, RAM, etc.
- Code reusability	
- Readability	including comments to document the code

41

To take away today

- Course overview
- First notion of *class* and *object*
- Objectives
- Main covered topics
- Recommend bibliography
- Assessment
- OOP web resourses

42